MÓDULO 06 · MACHINE LEARNING NA MÃO, COM NUMPY

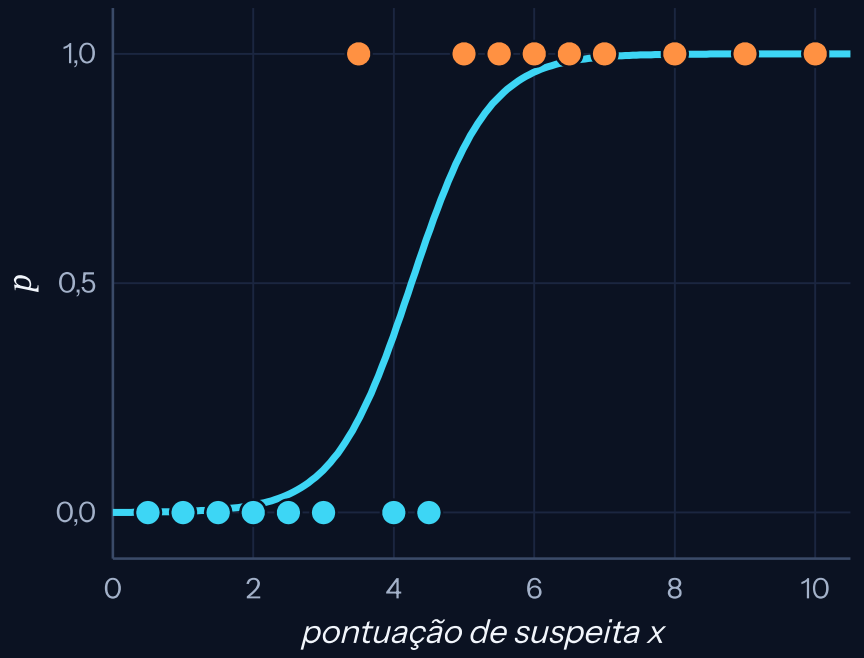
Aula 6.4

Regressão logística do zero

Sigmoide, entropia cruzada, gradiente da log-verossimilhança e fronteira de decisão, tudo em NumPy.

2 horas

Prática: classificador binário em ~40 linhas



Objetivos da aula

1

Explicar por que a regressão linear não serve para classificar em 0 ou 1

2

Transformar as entradas em uma probabilidade usando uma combinação linear e a função sigmoide

3

Calcular a entropia cruzada à mão e interpretar cada um dos seus termos

4

Deduzir o gradiente do custo e usá-lo para atualizar os parâmetros

5

Implementar, treinar e avaliar o classificador em NumPy e desenhar sua fronteira

ROTEIRO · 120 MINUTOS

Abertura	Teoria: sigmoide, custo, gradiente, fronteira	Quiz	Exercício guiado	Fechamento
6 min	73 min	2 × 3	24 min	11 min

O que ficou das aulas 6.2 e 6.3

MODELO DE REGRESSÃO

$$\hat{y} = X\beta$$

$\hat{y}$ ("y chapéu"): previsão, um número contínuo

X: matriz de dados, uma linha por amostra

β: coeficientes, o peso de cada característica

FUNÇÃO DE CUSTO

$$J(\beta) = \frac{1}{2m}\,\Sigma(\hat{y}_i - y_i)^2$$

y_i: valor real da amostra i; m: nº de amostras

Σ: soma de $i = 1$ até m

Erro quadrático médio; na 6.2, mínimo pela equação normal

GRADIENTE DESCENDENTE

$$\beta \leftarrow \beta - \alpha\nabla J(\beta)$$

$\nabla J(\beta)$: gradiente, direção em que o custo mais cresce

α: taxa de aprendizado, o tamanho do passo

$\leftarrow$: "recebe"; repetimos até J parar de cair

E se, em vez de um número, a resposta for uma categoria?

Classe 0: não é spam
Classe 1: é spam

Regressão e classificação

REGRESSÃO · AULAS 6.2 E 6.3

Prever o preço de um imóvel

Entradas: área, quartos, bairro

R$ 412.000

A saída é qualquer número real. O erro é uma distância: errar por R$ 5 mil é melhor que errar por R$ 50 mil.

→

CLASSIFICAÇÃO · HOJE

Decidir se uma mensagem é spam

Entradas: palavras suspeitas, número de links

0 ou 1

Só existem duas respostas. O modelo deve dizer qual é a mais provável e com que confiança.

Classificação binária

Classe — Cada categoria possível. Na classificação binária são duas: 0 e 1.

Rótulo y — A resposta correta de cada exemplo, conhecida no treino: $y \in \{0, 1\}$, isto é, y pertence ao conjunto $\{0, 1\}$.

Atributos x — Os números que descrevem o exemplo e influenciam a decisão.

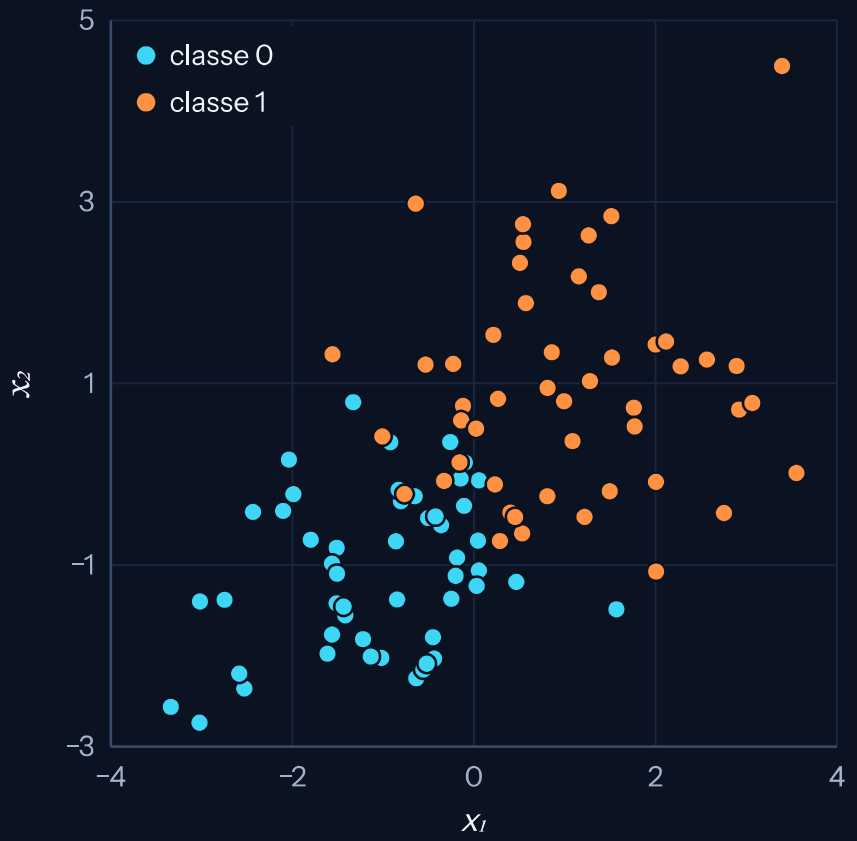
Duas saídas — Probabilidade p (confiança, entre 0 e 1) e classe prevista $\hat{y}$ (decisão, 0 ou 1).

Atenção: na regressão linear, $\hat{y}$ era um número qualquer. A partir de agora, $\hat{y}$ é a classe prevista.

PROBLEMA	CLASSE 1	CLASSE 0
E-mail	é spam	não é spam
Cliente	compra	não compra
Aluno	aprovado	reprovado
Transação	suspeita	normal

Convenção: classe 1 é o evento que queremos detectar, não a categoria “boa”.

O dataset da aula



100
amostras (m)

2
características

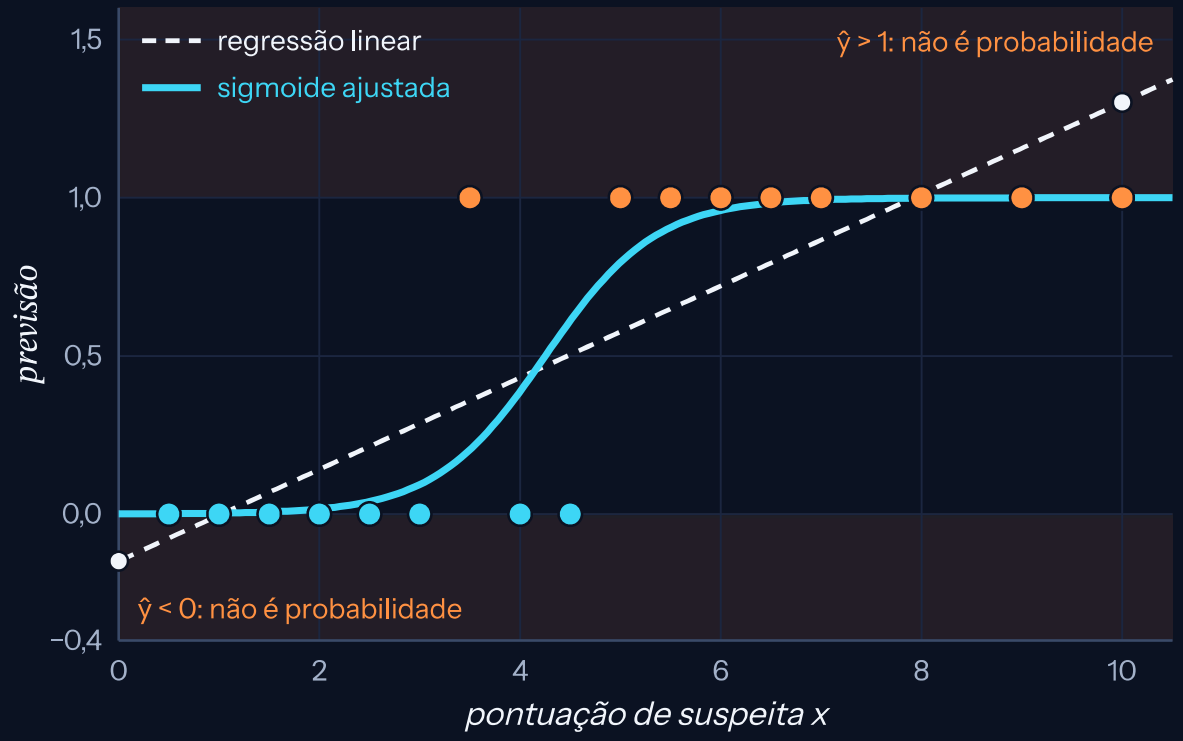
50/50
por classe

LINHA	x_1	x_2	y
0	−0,63	−2,25	0
1	−0,10	0,13	0
50	0,55	2,56	1
51	0,57	1,89	1

Cada ponto é uma amostra (x_1, x_2): 1ª e 2ª característica. A cor é o rótulo y.

Dados sintéticos gerados com NumPy, semente 42. Há sobreposição no centro: nenhuma reta acerta todos os pontos.

Por que não usar a regressão linear



$$\hat{y} = \beta_0 + \beta_1 x$$

x: pontuação de suspeita · $\hat{y}$: previsão numérica da reta · β_0: intercepto (valor de $\hat{y}$ em $x = 0$) · β_1: inclinação (variação de $\hat{y}$ por unidade de x)

Mínimos quadrados (aula 6.2) nestes 17 pontos: $\beta_0 = -0{,}149$ e $\beta_1 = 0{,}145$.

$x = 0 \rightarrow \hat{y} = -0{,}15$

$x = 10 \rightarrow \hat{y} = 1{,}30$

Uma probabilidade precisa ficar entre 0 e 1. A reta não respeita esse intervalo.

A curva em S, a sigmoide do próximo slide, transforma qualquer número real em um valor entre 0 e 1.

Função sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

σ

Sigma minúsculo: o nome da função sigmoide.

z

A entrada: qualquer número real, de $-\infty$ (menos infinito) a $+\infty$.

e

Constante de Euler, ≈ 2,71828, base do logaritmo natural.

e^{-z}

Exponencial de $-z$: sempre positiva.

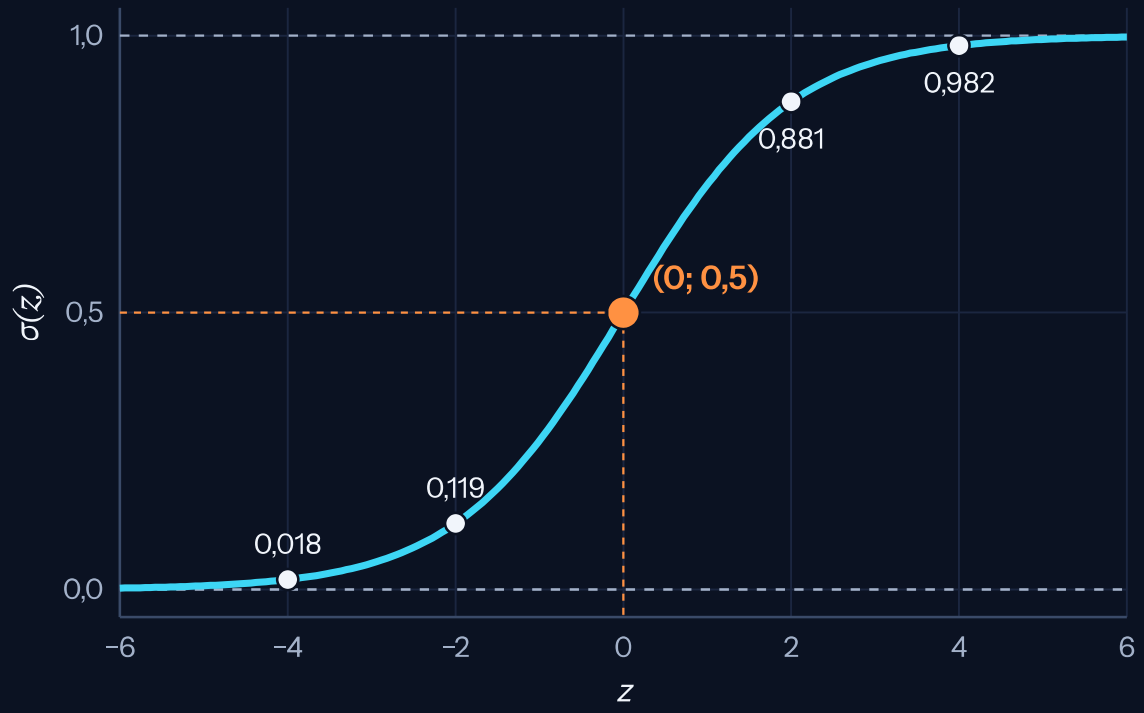
$1 + e^{-z}$

Denominador: sempre maior que 1.

$\sigma(z)$

Saída: 1 dividido por algo maior que 1, logo entre 0 e 1.

Comportamento da sigmoide



z	e^{-z}	$\sigma(z)$
−4	54,598	0,018
−2	7,389	0,119
0	**1**	**0,500**
2	0,135	0,881
4	0,018	0,982

z muito negativo: e^{-z} enorme, σ se aproxima de 0

$z = 0$: $e^{0} = 1$, logo $\sigma = 1/(1 + 1) = 0{,}5$

z muito positivo: e^{-z} quase 0, σ se aproxima de 1

Sigmoide em NumPy

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.array([-4, -2, 0, 2, 4])
print(sigmoid(z).round(3))
```

SAÍDA

```
[0.018 0.119 0.5   0.881 0.982]
```

np.exp eleva e a cada elemento; -z troca o sinal; 1 / (...) divide elemento a elemento; .round(3) arredonda para 3 casas. Vale para número, vetor ou matriz.

ESTABILIDADE NUMÉRICA

Com $z = -1000$, np.exp(1000) passa do maior float64 ($\approx 1{,}8 \times 10^{308}$) e vira inf, infinito, com o aviso RuntimeWarning: overflow encountered in exp. Depois, $p = 0$ exato gera $\log(0) = -\infty$ no custo.

Solução: limitar z com np.clip. Na saída, 7.12e-218 significa $7{,}12 \times 10^{-218}$: minúsculo, mas positivo.

```python
def sigmoid_estavel(z):
    z = np.clip(z, -500, 500)  # limita z antes da exponencial
    return 1 / (1 + np.exp(-z))

print(sigmoid_estavel(np.array([-1000.0, 0.0, 1000.0])))
```

SAÍDA

```
[7.12457641e-218 5.00000000e-001 1.00000000e+000]
```

Da entrada à probabilidade

ENTRADAS
x_1, x_2

→

COMBINAÇÃO LINEAR
$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2$

→

SIGMOIDE
$p = \sigma(z)$

→

PROBABILIDADE
$p \in (0, 1)$
entre 0 e 1

→

CLASSE
$\hat{y} \in \{0, 1\}$
só 0 ou 1

z

Combinação linear das entradas, antes da sigmoide: um escore, qualquer número real

β_0

Intercepto ou viés: o valor de z quando $x_1 = x_2 = 0$

β_1, β_2

Pesos da 1ª e da 2ª característica

x_1, x_2

Características da amostra usadas na classificação

β

Vetor de parâmetros $[\beta_0, \beta_1, \beta_2]$: o que o treino aprende

$p = P(y = 1 \mid x; \beta)$

Probabilidade estimada de a amostra ser da classe 1; "|" lê-se "dado": dados x e β

$\hat{y}$

Classe prevista ("y chapéu"), obtida comparando p com um limiar

Previsão passo a passo

PARÂMETROS ILUSTRATIVOS $\beta_0 = -2$ $\beta_1 = 0{,}5$ por palavra suspeita $\beta_2 = 1{,}0$ por link

Mensagem A

$x_1 = 3, x_2 = 2$

1. $z = -2 + 0{,}5 \cdot 3 + 1{,}0 \cdot 2 = 1{,}5$
2. $p = \frac{1}{1 + e^{-1{,}5}} = \frac{1}{1{,}223} = 0{,}818$
3. $0{,}818 \geq 0{,}5 \rightarrow \hat{y} = 1$, spam

Mensagem B

$x_1 = 1, x_2 = 0$

1. $z = -2 + 0{,}5 \cdot 1 + 1{,}0 \cdot 0 = -1{,}5$
2. $p = \frac{1}{1 + e^{1{,}5}} = \frac{1}{5{,}482} = 0{,}182$
3. $0{,}182 < 0{,}5 \rightarrow \hat{y} = 0$, não é spam

Várias amostras de uma vez: $z = X\beta$

$$\underset{\substack{1 \quad x_1 \quad x_2 \\ X \cdot (2,\ 3)}}{\begin{pmatrix} 1 & 3 & 2 \\ 1 & 1 & 0 \end{pmatrix}} \cdot \underset{B \cdot (3,)}{\begin{pmatrix} -2 \\ 0{,}5 \\ 1{,}0 \end{pmatrix}} = \underset{Z \cdot (2,)}{\begin{pmatrix} 1{,}5 \\ -1{,}5 \end{pmatrix}} \longrightarrow \sigma \longrightarrow \underset{P \cdot (2,)}{\begin{pmatrix} 0{,}818 \\ 0{,}182 \end{pmatrix}}$$

LINHAS DE X

Uma amostra por linha: A = [1, 3, 2], B = [1, 1, 0]. A 1ª coluna vale sempre 1 e multiplica β_0: é assim que o intercepto entra.

PRODUTO XB

Cada linha vezes β, somando: 1 × (−2) + 3 × 0,5 + 2 × 1,0 = 1,5. Um z por amostra.

NO NUMPY

```
z = X @ beta
p = sigmoid(z)
```

@: produto matricial. A sigmoide age em cada elemento.

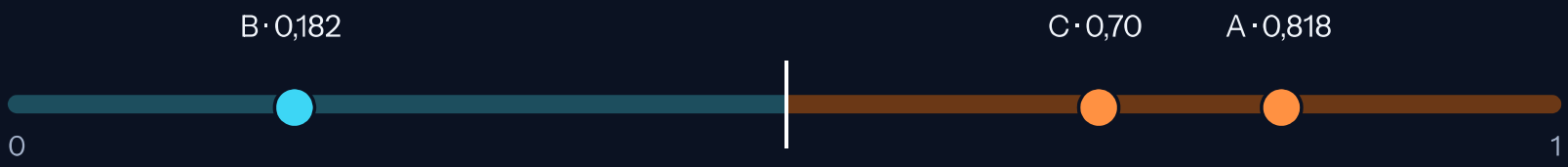
Limiar de classificação

$$\hat{y} = \begin{cases} 1 \text{ se } p \geq t \\ 0 \text{ se } p < t \end{cases}$$

t: limiar, escolhido conforme o problema

t = 0,5

padrão: classe mais provável



t = 0,9

filtro cauteloso



Probabilidade estimada não é a mesma coisa que classe prevista.

• classe 0 • classe 1

z, p e $\hat{y}$: números diferentes

	z ESCORE	p PROBABILIDADE	$\hat{y}$ CLASSE PREVISTA	y RÓTULO VERDADEIRO
O QUE É	combinação linear das entradas	chance estimada da classe 1	decisão do modelo	resposta correta, vem dos dados
VALORES	qualquer número real	entre 0 e 1	0 ou 1	0 ou 1
MENSAGEM A	1,5	0,818	1	1
MENSAGEM B	−1,5	0,182	0	0
NO CÓDIGO	`X @ beta`	`sigmoid(z)`	`(p ≥ 0.5).astype(int)`	`y`

O treino compara p com y.

A acurácia compara $\hat{y}$ com y.

$z = 0 \Leftrightarrow p = 0{,}5$: onde a decisão muda.

Quiz rápido · 1ª rodada

1

Por que a regressão logística é usada para classificação?

2

O que representa a saída da sigmoide?

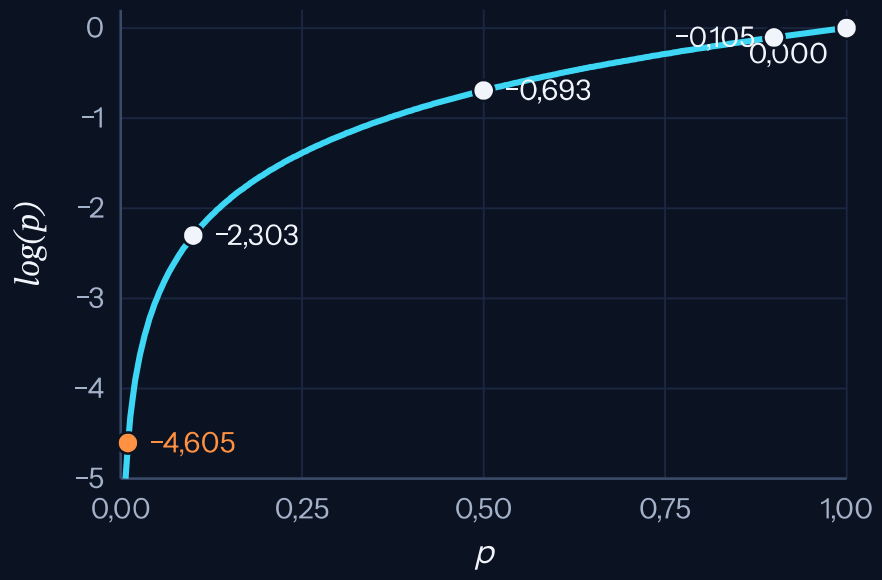
3

O que acontece quando $z = 0$?

4

Uma probabilidade de 0,7 corresponde a qual classe com limiar 0,5?

Logaritmo natural: o que vamos usar



p	$\log(p)$
1	0
0,9	−0,105
0,5	−0,693
0,1	−2,303
0,01	−4,605

log = logaritmo natural (base e), o np.log

Entre 0 e 1, o log é negativo; perto de 0, despenca ($\to -\infty$)

$\log(a \cdot b) = \log a + \log b$ · crescente · $(\log p)' = 1/p$

Entropia cruzada binária

Mede quão pouca probabilidade o modelo deu à classe correta. Quanto menor, melhor. O sinal de menos deixa J positivo, porque o log de um número entre 0 e 1 é negativo.

$$J(\beta) = -\frac{1}{m}\sum_{i=1}^{m}\left[\, y_i \log(p_i) + (1 - y_i)\log(1 - p_i)\,\right]$$

$J(\beta)$
Custo: um número para o conjunto de parâmetros

β
Parâmetros do modelo; J muda quando β muda

m
Número de amostras; $1/m$ tira a média

Σ
Somatório de $i = 1$ até m

y_i
Classe verdadeira da amostra i: 0 ou 1

p_i
Probabilidade estimada da classe 1 para a amostra i

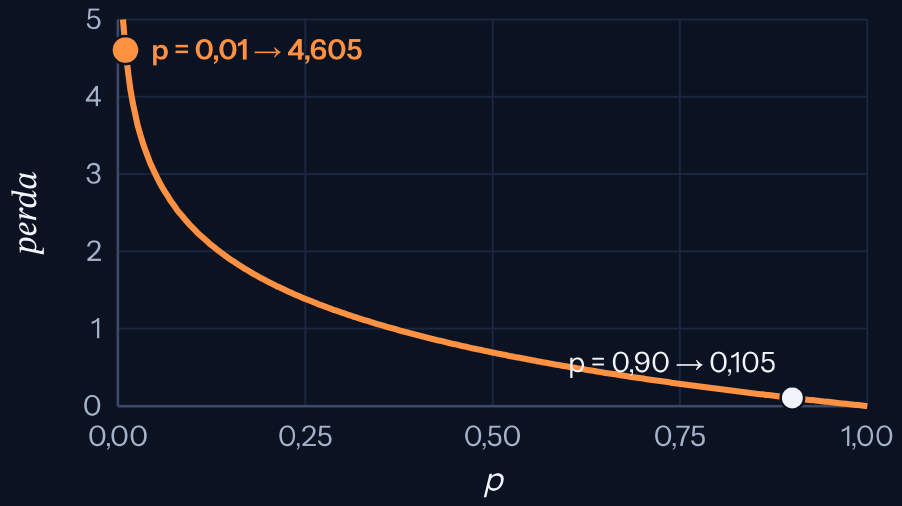
log
Logaritmo natural (ln), o np.log

$1 - y_i$
Vale 1 só quando a classe é 0: liga o 2º termo, que usa $1 - p_i$, a probabilidade da classe 0

Entropia cruzada: os dois casos

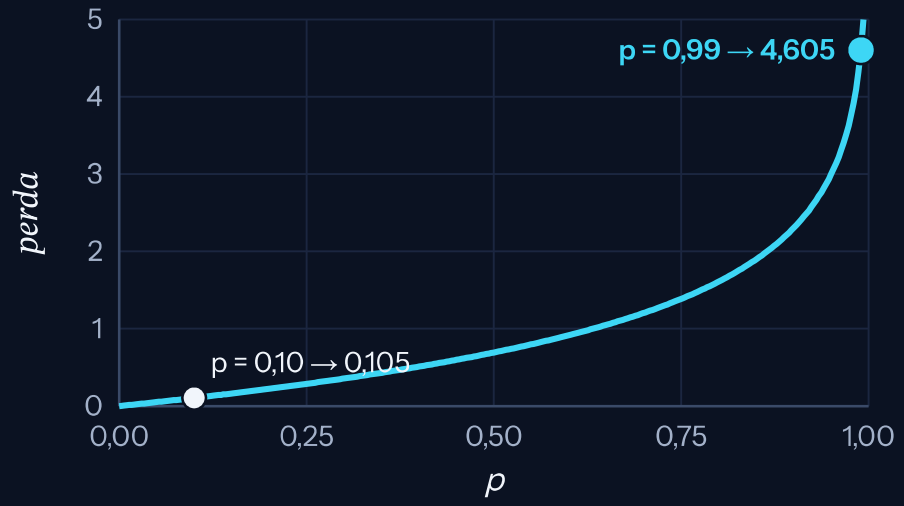
Se $y = 1$ $\text{Perda} = -\log(p)$

$1 \cdot \log(p) + (1 - 1) \cdot \log(1 - p)$: o segundo termo vira zero.



Se $y = 0$ $\text{Perda} = -\log(1 - p)$

$0 \cdot \log(p) + (1 - 0) \cdot \log(1 - p)$: o primeiro termo vira zero.



Entropia cruzada: exemplo resolvido

AMOSTRA	Y_I	P_I	TERMO ATIVO	PERDA
1	1	0,90	$-\log(0{,}90)$	0,105
2	1	0,60	$-\log(0{,}60)$	0,511
3	0	0,20	$-\log(1-0{,}20)$	0,223
4	**0**	**0,95**	$-\log(1-0{,}95)$	**2,996**
Soma das perdas				3,835

$$J = \frac{3{,}835}{4} = 0{,}959$$

Perda Erro de uma amostra

Custo J Média das perdas do conjunto: é o que o treino minimiza

A amostra 4, confiante e errada, responde por 78% do custo.

Verossimilhança

PROBABILIDADE

β fixo. Qual a chance de $y = 1$ para esta amostra?

VEROSSIMILHANÇA

Rótulos observados fixos. Quão bem estes β explicam o que vimos?

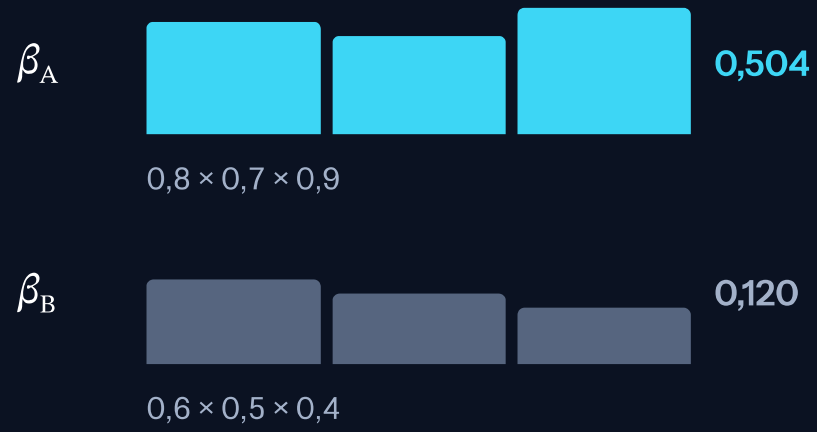
$$P(y_i \mid x_i; \beta) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

Lê-se: probabilidade de y_i dado x_i, com parâmetros β. Como todo número elevado a 0 vale 1: se $y_i = 1$, sobra p_i; se $y_i = 0$, sobra $1 - p_i$. Nos dois casos, a probabilidade dada à classe correta.

$$L(\beta) = \prod_{i=1}^{m} P(y_i \mid x_i; \beta)$$

Π: produtório, multiplica os termos de $i = 1$ até m. Com amostras independentes (uma não influencia a outra), a probabilidade do conjunto é o produto.

PROBABILIDADE DADA À CLASSE CORRETA



β_A explica melhor os rótulos observados.

Da log-verossimilhança à entropia cruzada

PRODUTO VIRA SOMA

$\log(0{,}8 \cdot 0{,}7 \cdot 0{,}9) = -0{,}223 - 0{,}357 - 0{,}105 = -0{,}685$

EVITA UNDERFLOW

$0{,}9^{1000} \approx 1{,}7 \times 10^{-46}$: com muitas amostras o produto é arredondado para 0. A soma dos logs, −105,4, não.

MESMO MÁXIMO

log é crescente: o β que maximiza L maximiza log L

$$\ell(\beta) = \log L(\beta) = \Sigma\,[\, y_i \log p_i + (1 - y_i) \log(1 - p_i)\,]$$

$$J(\beta) = -\,\frac{1}{m}\,\ell(\beta)$$

maximizar ℓ ⇔ minimizar J (⇔: "equivale a")

SUBIDA EM ℓ

$$\nabla \ell = X^{\mathrm{T}}(y - p)$$

$$\beta \leftarrow \beta + \alpha \cdot \nabla \ell / m$$

X^{T}: transposta de X. Dedução nos próximos slides.

DESCIDA EM J

$$\nabla J = X^{\mathrm{T}}(p - y)/m$$

$$\beta \leftarrow \beta - \alpha \cdot \nabla J$$

Os dois sinais trocam juntos: a atualização é a mesma.

Gradiente descendente na regressão logística

$$\beta \leftarrow \beta - \alpha \nabla J(\beta)$$

β

Vetor de parâmetros $[\beta_0, \beta_1, \beta_2]$

α

Taxa de aprendizado: o tamanho do passo (ex.: 0,1)

∇

Nabla, operador gradiente: deriva em relação a cada parâmetro

J

Função de custo: a entropia cruzada

$\nabla J(\beta)$

Vetor $[\partial J/\partial \beta_0, \partial J/\partial \beta_1, \partial J/\partial \beta_2]$

$\partial J/\partial \beta_j$

Derivada parcial: quanto J muda se só β_j variar um pouco (j = 0, 1, 2)

PARA OBTER ∇J 1 · Derivada da sigmoide → 2 · Regra da cadeia → 3 · Forma vetorizada

Derivada da sigmoide

1 $$\sigma(z) = (1 + e^{-z})^{-1}$$

Com $u = 1 + e^{-z}$, escrevemos $1/u$ como u^{-1}

2 $$\sigma'(z) = -u^{-2} \cdot u' = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Regra da cadeia: $(u^{-1})' = -u^{-2}u'$. E $u' = -e^{-z}$, pois a derivada de e^{-z} é $e^{-z}(-1)$

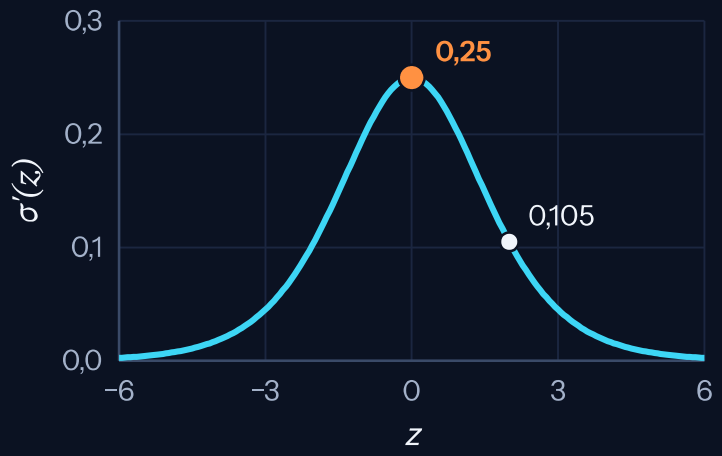
3 $$= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

Separamos o quadrado em duas frações; a primeira é $\sigma(z)$

4 $$\frac{e^{-z}}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - \sigma(z)$$

Somamos e subtraímos 1 no numerador

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$



Máximo 0,25 em $z = 0$. Em $z = 2$: 0,881 × 0,119 ≈ 0,105.

Com $p = \sigma(z)$ já calculado: $\sigma'(z) = p(1 - p)$.

Gradiente da entropia cruzada (1/2): três derivadas

$J_i = -[y \log p + (1-y)\log(1-p)]$ perda de uma amostra (omitimos o índice i em y e p)

$\beta_j \rightarrow z \rightarrow p = \sigma(z) \rightarrow J_i$

$$\partial J_i/\partial \beta_j = \partial J_i/\partial p \cdot \partial p/\partial z \cdot \partial z/\partial \beta_j$$

1 · $\partial J_i/\partial P$

$(\log p)' = 1/p$ e $(\log(1-p))' = -1/(1-p)$

$$= -\frac{y}{p} + \frac{1-y}{1-p}$$

Mesmo denominador $p(1-p)$:

$$= \frac{-y(1-p) + (1-y)p}{p(1-p)} = \frac{p-y}{p(1-p)}$$

Numerador: $-y + yp + p - yp = p - y$

2 · $\partial P/\partial Z$

$$p(1-p)$$

Derivada da sigmoide, do slide anterior

3 · $\partial Z/\partial B_J$

$$x_j$$

Em $z = \beta_0 \cdot 1 + \beta_1 x_1 + \beta_2 x_2$, só o termo de β_j depende dele. Para o intercepto, $x_0 = 1$.

Gradiente da entropia cruzada (2/2): produto e média

$$\partial J_i/\partial \beta_j = \frac{p - y}{\cancel{p(1 - p)}} \cdot \cancel{p(1 - p)} \cdot x_j = (p - y)\, x_j$$

Erro da previsão × característica. O fator $p(1 - p)$ se cancela.

$$\partial J/\partial \beta_j = \frac{1}{m} \Sigma_i (p_i - y_i)\, x_{ij}$$

J é a média das perdas, então a derivada é a média das derivadas. x_{ij}: característica j da amostra i.

CHECAGEM NUMÉRICA

Amostra com $x = 2$, $y = 1$, $p = 0{,}5$:

para β_0: $(0{,}5 - 1)\cdot 1 = -0{,}5$ · para β_1: $(0{,}5 - 1)\cdot 2 = -1{,}0$

INTERPRETAÇÃO

Se $p_i = y_i$ em todas as amostras, o gradiente é zero: não há o que corrigir.

Por que não o erro quadrático

CENÁRIO $y = 1$ e o modelo responde $p = 0{,}01$: confiante e errado

Erro quadrático

PERDA J_I

$$(y - p)^2 = 0{,}980$$

$\partial J_I / \partial Z$ (REGRA DA CADEIA)

$$2(p - y) \cdot p(1 - p) = -0{,}0196$$

Quase zero: $p(1 - p)$ não se cancela e some quando a sigmoide satura.

Entropia cruzada

PERDA J_I

$$-\log(p) = 4{,}605$$

$\partial J_I / \partial Z$

$$p - y = -0{,}990$$

Forte e proporcional ao erro.

Convexa = formato de tigela, com um único vale. A entropia cruzada é convexa em β: o gradiente descendente não fica preso em mínimos locais. Com a sigmoide dentro do erro quadrático, essa garantia se perde.

Gradiente vetorizado

$$\nabla J(\beta) = \frac{1}{m} X^{\mathrm{T}}(p - y)$$

X^{T} (3 × 80) · $p - y$ (80,) = ∇J (3,)

```
gradiente = (X.T @ (p - y)) / m

beta = beta - alpha * gradiente
```

`X.T`	transposta: (m, 3) vira (3, m)
`@`	produto matricial: faz a soma Σ sobre as amostras
`p - y`	vetor de erros, um por amostra
`/ m`	média sobre as amostras
`- alpha *`	passo de tamanho α contra o gradiente

REGRESSÃO LINEAR · 6.3

$$\nabla J = X^{\mathrm{T}}(X\beta - y)/m$$

com $J = \Sigma(\hat{y} - y)^2/(2m)$

REGRESSÃO LOGÍSTICA · HOJE

$$\nabla J = X^{\mathrm{T}}(\sigma(X\beta) - y)/m$$

com J = entropia cruzada

Mesma forma, X^{T}(previsão − rótulo)/m, graças ao cancelamento de $p(1-p)$.

X: m linhas (amostras) e n + 1 colunas (n características + a coluna de 1s). Aqui n = 2.

Exemplo resolvido: primeira iteração (1/2)

$x = [0, 1, 2]$ $y = [0, 0, 1]$ $\beta_0 = \beta_1 = 0$ $\alpha = 0{,}1$ $X = [[1, 0], [1, 1], [1, 2]]$

AMOSTRA	X	Y	1 · $Z = B_0 + B_1X$	2–3 · $P = \Sigma(Z)$	4 · $P - Y$	5 · PERDA
1	0	0	0 + 0·0 = 0	0,5	0,5	−log(1 − 0,5) = 0,6931
2	1	0	0 + 0·1 = 0	0,5	0,5	−log(1 − 0,5) = 0,6931
3	2	1	0 + 0·2 = 0	0,5	−0,5	−log(0,5) = 0,6931

$$J(\beta) = \frac{0{,}6931 + 0{,}6931 + 0{,}6931}{3} = 0{,}6931 = \ln 2$$

Com $\beta = 0$ o modelo responde 0,5 para tudo. Use esse valor para testar seu código.

Exemplo resolvido: primeira iteração (2/2)

6 · DERIVADAS

$$\partial J/\partial \beta_0 = (0{,}5 + 0{,}5 - 0{,}5)/3 = 0{,}1667$$

$$\partial J/\partial \beta_1 = (0{,}5\cdot 0 + 0{,}5\cdot 1 - 0{,}5\cdot 2)/3 = -0{,}1667$$

Em forma matricial: $X^{\top}(p - y)/3$ = [0,5; −0,5]/3, a mesma conta.

7 · ATUALIZAÇÃO

$$\beta_0 = 0 - 0{,}1 \cdot 0{,}1667 = -0{,}0167$$

$$\beta_1 = 0 - 0{,}1 \cdot (-0{,}1667) = 0{,}0167$$

8–9 · ANTES E DEPOIS	ANTES	DEPOIS
β_0, β_1	0, 0	−0,0167, 0,0167
p amostra 1 (y = 0)	0,5000	0,4958 ↓
p amostra 2 (y = 0)	0,5000	0,5000
p amostra 3 (y = 1)	0,5000	0,5042 ↑
Custo J	**0,6931**	**0,6876**

10 · INTERPRETAÇÃO

O custo caiu. A classe 1 ganhou probabilidade e a classe 0 perdeu.

APÓS 1000 ITERAÇÕES

β_0 = −5,185 · β_1 = 3,634

J = 0,105 · fronteira em $x \approx 1{,}43$

Fronteira de decisão

$p \geq 0{,}5 \iff \sigma(z) \geq 0{,}5 \iff z \geq 0$ porque σ é crescente e $\sigma(0) = 0{,}5$

Fronteira: $\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0$

$$x_2 = -\frac{\beta_0 + \beta_1 x_1}{\beta_2} = -\frac{\beta_0}{\beta_2} + \left(-\frac{\beta_1}{\beta_2}\right) x_1$$ se $\beta_2 \neq 0$

INCLINAÇÃO $-B_1/B_2$

Quanto x_2 muda, ao longo da fronteira, quando x_1 aumenta 1

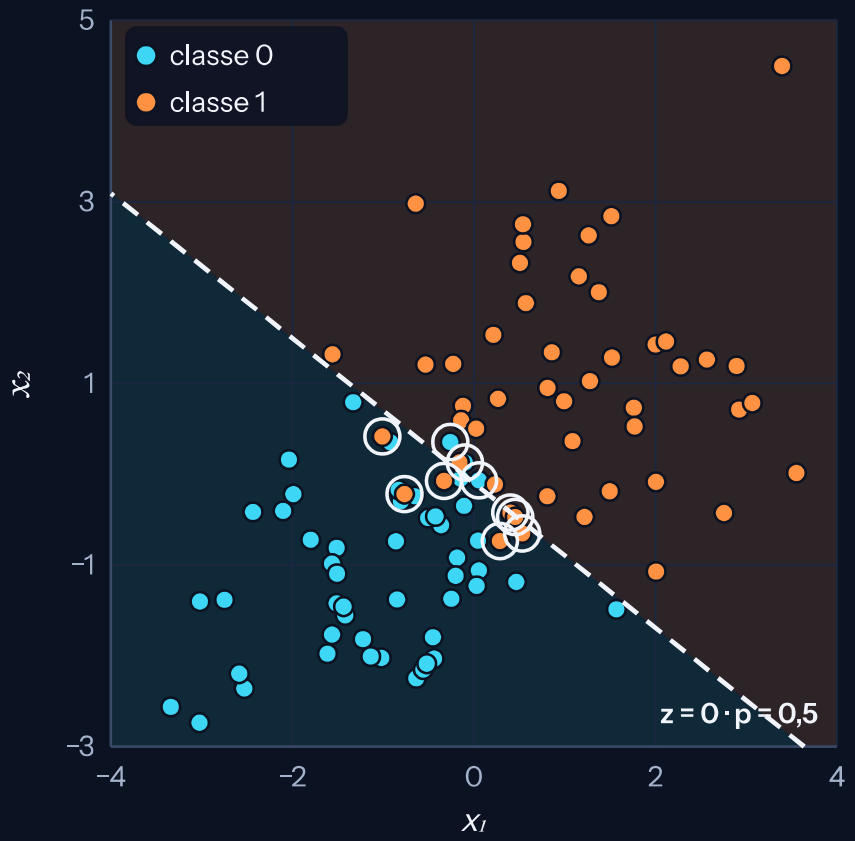
INTERCEPTO $-B_0/B_2$

Onde a reta cruza o eixo x_2, isto é, em $x_1 = 0$

DOIS LADOS

$z > 0$: classe 1. $z < 0$: classe 0. Com características originais, a fronteira é sempre uma reta.

Fronteira aprendida nos dados da aula



PARÂMETROS APRENDIDOS

$$\beta = [0{,}257;\ 2{,}116;\ 2{,}653]$$

RETA DA FRONTEIRA

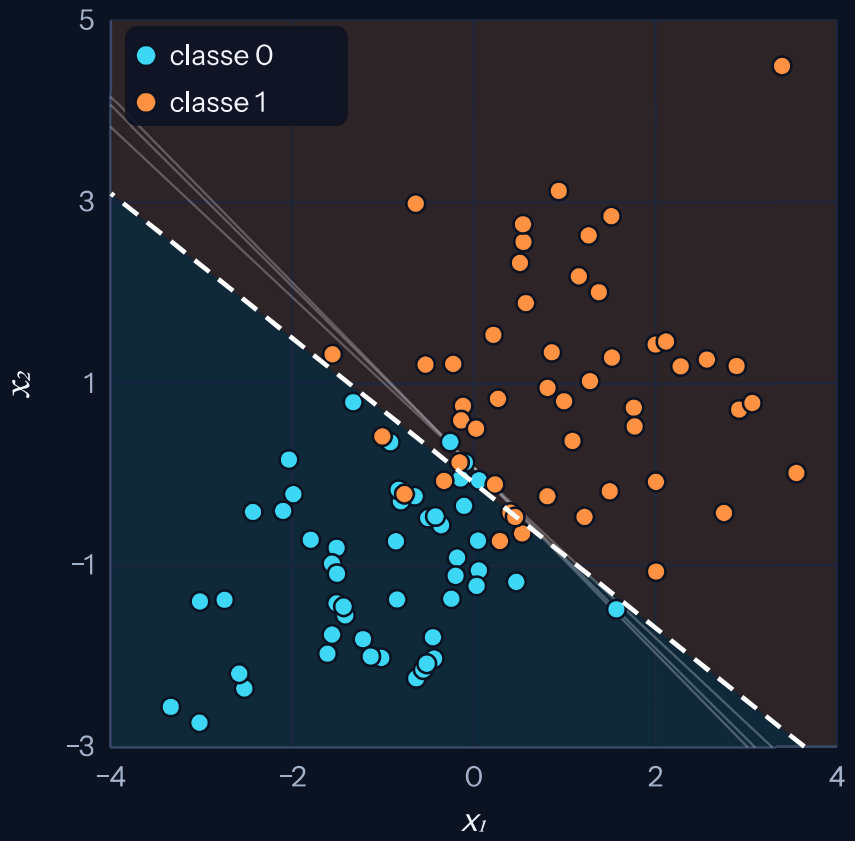
$$x_2 \approx -0{,}10 - 0{,}80 \cdot x_1$$

0,900
acurácia no treino · 72/80

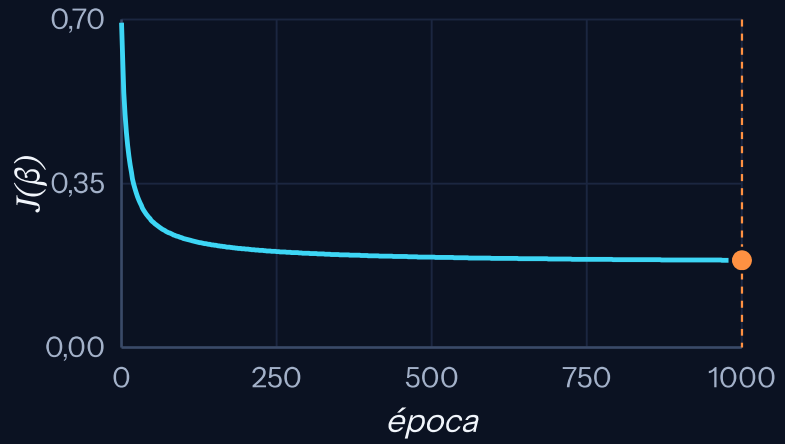
0,900
acurácia no teste · 18/20

Anel branco: ponto classificado errado. Todos estão na faixa de sobreposição das classes.

Fronteira durante o treinamento



ÉPOCA	B_0	B_1	B_2	J(B)
1000	0,257	2,116	2,653	0,1862



Reproduzir treino

Linhas tênues: fronteiras nas épocas 2, 10 e 50.

Quiz rápido · 2ª rodada

1. O que significa a entropia cruzada?
2. Por que precisamos calcular o gradiente?
3. O que acontece se a taxa de aprendizado for muito alta?
4. O que representa a fronteira de decisão?
5. Como a regressão logística aprende os coeficientes?

Passo 1 · Gerar e visualizar os dados

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)  # semente: os mesmos dados sempre
m_por_classe = 50
X0 = rng.normal(loc=[-1, -1], scale=1.2, size=(m_por_classe, 2))  # classe 0
X1 = rng.normal(loc=[1, 1], scale=1.2, size=(m_por_classe, 2))  # classe 1

X_dados = np.vstack([X0, X1])  # (100, 2) características
y = np.concatenate([np.zeros(m_por_classe),
                    np.ones(m_por_classe)])  # (100,) rótulos 0/1

plt.scatter(X_dados[y == 0, 0], X_dados[y == 0, 1], label="classe 0")
plt.scatter(X_dados[y == 1, 0], X_dados[y == 1, 1], label="classe 1")
plt.xlabel("x1"); plt.ylabel("x2"); plt.legend(); plt.show()
```

ANTES DE DIGITAR

Semente 42: todos geram os mesmos números dos slides.

50 pontos por classe, centros (−1, −1) e (1, 1), desvio 1,2.

X_dados: características (100, 2). y: rótulos (100,).

CONFIRA

X_dados.shape é (100, 2) e y.sum() é 50.

Passo 2 · Separar os dados e montar a matriz X

```
idx = rng.permutation(len(y))  # embaralha os índices
corte = int(0.8 * len(y))  # 80% treino, 20% teste
treino, teste = idx[:corte], idx[corte:]

uns = np.ones((X_dados.shape[0], 1))  # coluna de 1s: multiplica β0
X = np.hstack([uns, X_dados])  # (100, 3) = [1, x1, x2]

X_treino, y_treino = X[treino], y[treino]
X_teste, y_teste = X[teste], y[teste]
print(X_treino.shape, y_treino.shape)

beta = np.zeros(X.shape[1])  # [β0, β1, β2] = [0, 0, 0]
print(beta)
```

SAÍDA

```
(80, 3) (80,)
[0. 0. 0.]
```

ANTES DE DIGITAR

80 amostras para treinar, 20 trancadas para o teste final.

A coluna de 1s faz X @ beta incluir o intercepto:

$$[1, x_1, x_2] \cdot [\beta_0, \beta_1, \beta_2] = z$$

Sem ela, a fronteira é obrigada a passar pela origem.

Passo 3 · Sigmoide, custo e gradiente

```python
def sigmoid(z):
    return 1 / (1 + np.exp(-z))  # σ(z), aplicada elemento a elemento

def custo(p, y, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)  # evita log(0)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

def gradiente(X, p, y):
    m = len(y)
    return X.T @ (p - y) / m  # ∇J(β): uma derivada por β
```

CADA FUNÇÃO, UMA FÓRMULA

sigmoid

$$\sigma(z) = 1/(1 + e^{-z})$$

custo

$$J = -(1/m)\,\Sigma[y \log p + (1 - y) \log(1 - p)]$$

gradiente

$$\nabla J = X^{T}(p - y)/m$$

CONFIRA

custo(np.full(80, 0.5), y_treino) dá 0,6931 = ln 2.

Passo 4 · Treinar e guardar o histórico

```python
alpha, epocas = 0.1, 1000
beta = np.zeros(X_treino.shape[1])
historico = []

for epoca in range(epocas):
    z = X_treino @ beta  # combinação linear, (80,)
    p = sigmoid(z)  # probabilidades, (80,)
    historico.append(custo(p, y_treino))  # guarda J(β) desta época
    beta = beta - alpha * gradiente(X_treino, p, y_treino)
    if epoca % 200 == 0:
        print(f"época {epoca:4d}  custo {historico[-1]:.4f}")

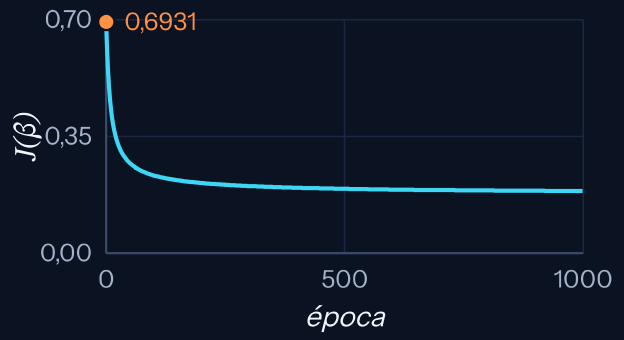
print("beta:", beta.round(3))
```

SAÍDA REAL

Com mais épocas, o custo se aproxima de ≈ 0,183.

SAÍDA

```
época    0  custo 0.6931
época  200  custo 0.2102
época  400  custo 0.1960
época  600  custo 0.1905
época  800  custo 0.1878
beta: [0.257 2.116 2.653]
```



Passo 5 · Probabilidades, classes e acurácia

```
p_teste = sigmoid(X_teste @ beta)  # probabilidades da classe 1
y_pred = (p_teste ≥ 0.5).astype(int)  # limiar 0,5 → classes 0/1

acc_teste = np.mean(y_pred == y_teste)  # fração de acertos
acc_treino = np.mean((sigmoid(X_treino @ beta) ≥ 0.5) == y_treino)

print(p_teste[:5].round(3), y_pred[:5], y_teste[:5].astype(int))
print(f"acurácia treino {acc_treino:.3f} | teste {acc_teste:.3f}")
```

SAÍDA

```
[1.    0.954 0.79  0.006 0.881] [1 1 1 0 1] [1 1 1 0 1]
acurácia treino 0.900 | teste 0.900
```

COMO LER

acurácia = acertos / amostras

Teste: 18 acertos em 20 amostras = 0,900.

0,790 e 0,954 viram a mesma classe 1: a classe descarta a confiança.

REGRA

Acurácia se calcula sobre classes, nunca sobre probabilidades.

Passo 6 · Desenhar a fronteira e a curva de custo

```
b0, b1, b2 = beta
x1 = np.linspace(X_dados[:, 0].min(), X_dados[:, 0].max(), 100)
x2 = -(b0 + b1 * x1) / b2  # reta onde z = 0 (p = 0,5)

plt.scatter(X_dados[y == 0, 0], X_dados[y == 0, 1], label="classe 0")
plt.scatter(X_dados[y == 1, 0], X_dados[y == 1, 1], label="classe 1")
plt.plot(x1, x2, "k--", label="fronteira p = 0,5")
plt.legend(); plt.show()

plt.plot(historico)  # curva de custo
plt.xlabel("época"); plt.ylabel("J(β)"); plt.show()
```

ANTES DE DIGITAR

A reta é o conjunto de pontos onde $z = 0$:

$$x_2 = -(\beta_0 + \beta_1 x_1)/\beta_2$$

Com o β aprendido: $x_2 \approx -0{,}10 - 0{,}80 x_1$.

CONFIRA

O desenho deve coincidir com o do slide “Fronteira aprendida”.

Erros comuns (1/2)

1 · Confundir com regressão linear

Causa: tratar z como a resposta. z é um escore; a previsão é $p = \sigma(z)$.

```
✗ custo = np.mean((X @ beta - y) ** 2)
```

```
✓ p = sigmoid(X @ beta); custo(p, y)
```

2 · Probabilidade tratada como classe

Causa: devolver p onde se espera 0 ou 1.

```
✗ def predict(self, X): return self.predict_proba(X)
```

```
✓ return (self.predict_proba(X) ≥ 0.5).astype(int)
```

3 · Esquecer a sigmoide

Causa: copiar o laço da 6.3. Valores fora de (0, 1) geram nan no log.

```
✗ p = X_treino @ beta
```

```
✓ p = sigmoid(X_treino @ beta)
```

4 · Sinal errado na atualização

Causa: somar o gradiente. Sintoma: o custo sobe a cada época.

```
✗ beta = beta + alpha * grad
```

```
✓ beta = beta - alpha * grad
```

5 · Esquecer a coluna do intercepto

Causa: usar X_dados direto. A fronteira fica presa à origem.

```
✗ X = X_dados
```

```
✓ X = np.hstack([np.ones((m, 1)), X_dados])
```

6 · Entropia cruzada incompleta

Causa: esquecer o termo da classe 0 ou o sinal de menos.

```
✗ -np.mean(y * np.log(p))
```

```
✓ -np.mean(y*np.log(p) + (1-y)*np.log(1-p))
```

Erros comuns (2/2)

7 · log(0) no custo

Causa: sigmoide saturada devolve 0 ou 1 exatos; log vira $-\infty$.

```
✗ np.log(p)  # p = 0.0 → -inf
```

```
✓ p = np.clip(p, 1e-12, 1 - 1e-12)
```

8 · Divergência despercebida

Causa: olhar só o β final. Nos dados da aula, com $\alpha = 100$, J sobe em 483 das 1000 épocas.

```
✗ print(beta)  # e nada mais
```

```
✓ plt.plot(historico); reduzir alpha
```

9 · Dimensões incompatíveis

Causa: y (80, 1) com p (80,): p - y vira (80, 80) sem erro.

```
✗ y = y.reshape(-1, 1)
```

```
✓ y e p com formato (m,); conferir .shape
```

10 · Acurácia sobre probabilidades

Causa: p quase nunca é exatamente 0 ou 1.

```
✗ np.mean(p == y)
```

```
✓ np.mean((p ≥ 0.5) == y)
```

11 · Fronteira incompatível com β

Causa: dividir por β_1 ou trocar a ordem. Teste: na reta, $p = 0{,}5$.

```
✗ x2 = -(b0 + b1 * x1) / b1
```

```
✓ x2 = -(b0 + b1 * x1) / b2
```

12 · Avaliar só no treino

Causa: o treino é otimista. O teste estima dados novos.

```
✗ modelo.score(X_treino, y_treino)
```

```
✓ modelo.score(X_teste, y_teste)
```

Desafio: classificador em ~40 linhas

Implemente a classe RegressaoLogistica usando só NumPy e treine-a por gradiente descendente.

`__init__()`	taxa de aprendizado e épocas
`fit(X, y)`	treina e guarda o histórico de custo
`predict_proba(X)`	probabilidades da classe 1
`predict(X)`	classes 0/1 com limiar 0,5
`score(X, y)`	acurácia

DEPOIS DE IMPLEMENTAR

1 Treinar nos dados da aula · 2 Visualizar os dados · 3 Prever · 4 Calcular a acurácia · 5 Desenhar a curva de custo · 6 Desenhar a fronteira aprendida

CRITÉRIOS DE ACEITE

✓ len(historico) igual ao número de épocas

✓ Primeiro custo = ln 2 ≈ 0,6931 (β começa em zero)

✓ Último custo menor que o primeiro

✓ predict_proba em (0, 1); predict só 0 e 1

✓ Acurácia no teste ≥ 0,85

✓ Fronteira desenhada a partir do β aprendido

Arquivo: 6.4_desafio.py · correção no início da aula 6.5

Do exercício guiado à classe

PASSO	EXERCÍCIO GUIADO	NA CLASSE
2	`np.hstack([uns, X_dados])`	`_com_intercepto(X)`
3	`sigmoid(z)`	`_sigmoid(z)`
3	`custo(p, y)`	`_custo(p, y)`
4	`for epoca in range(epocas): ...`	`fit(X, y)`
5	`sigmoid(X_teste @ beta)`	`predict_proba(X)`
5	`(p ⩾ 0.5).astype(int)`	`predict(X)`
5	`np.mean(y_pred == y_teste)`	`score(X, y)`

O QUE MUDA

`self`: o próprio objeto. `self.beta` e `self.historico` guardam o que `fit` aprendeu.

`self.alpha` e `self.epocas` vêm do `__init__`.

Métodos com `_` são auxiliares internos.

`fit` termina com `return self`.

Estrutura inicial do desafio

```python
import numpy as np

class RegressaoLogistica:
    def __init__(self, alpha=0.1, epocas=1000):
        # TODO 1: guardar alpha e epocas;
        #         beta = None, historico = []
        ...

    def _sigmoid(self, z):
        # TODO 2: 1 / (1 + e^(-z)), com np.clip em z
        ...

    def _com_intercepto(self, X):
        # TODO 3: coluna de 1s à esquerda de X
        ...

    def _custo(self, p, y, eps=1e-12):
        # TODO 4: entropia cruzada média (clip em p)
        ...
```

```python
    def fit(self, X, y):
        # TODO 5: Xb, beta = zeros, historico = []
        # TODO 6: repetir self.epocas vezes:
        #   p → custo → gradiente → atualização
        return self

    def predict_proba(self, X):
        # TODO 7: probabilidades da classe 1
        ...

    def predict(self, X, limiar=0.5):
        # TODO 8: probabilidades → classes 0/1
        ...

    def score(self, X, y):
        # TODO 9: fração de previsões corretas
        ...
```

Dicionário de símbolos (1/2): dados e modelo

x, x_j	característica; a j-ésima característica
$x_0 = 1$	entrada fixa que multiplica β_0
X^T	transposta de X (X.T)
y, y_i	rótulo verdadeiro, 0 ou 1
i, j	índice da amostra; índice do parâmetro
β_0	intercepto ou viés
z	escore linear $X\beta$, qualquer número real
e	constante de Euler ≈ 2,71828
$\hat{y}$	classe prevista 0/1 (na reg. linear: previsão numérica)
$\mid$	"dado": $P(y = 1 \mid x)$
$(0, 1)$	intervalo: entre 0 e 1

x_{ij}	característica j da amostra i
X	matriz $(m, n{+}1)$, linhas $[1, x_1, \ldots, x_n]$
@	produto matricial no NumPy
m, n	nº de amostras; nº de características
β	vetor de parâmetros aprendidos
β_j	peso da característica j
$\sigma(z)$	sigmoide: $1/(1 + e^{-z})$
p, p_i	probabilidade estimada da classe 1
t	limiar de classificação (padrão 0,5)
$\in$	"pertence a"
$\{0, 1\}$	conjunto: só 0 ou 1

Dicionário de símbolos (2/2): custo e otimização

$\log$	logaritmo natural (ln), np.log
$J(\beta)$	custo: média das perdas (entropia cruzada)
Π	produtório
$\ell(\beta)$	log-verossimilhança; $J = -\ell / m$
$\partial J / \partial \beta_j$	derivada parcial em relação a β_j
$\nabla J(\beta)$	gradiente: $X^{\top}(p - y)/m$
$\leftarrow$	“recebe”: atualização de valor
$\rightarrow$	“se aproxima de” ou “leva a”
$\approx$	aproximadamente
J_i	perda de uma amostra
Σ	somatório
$L(\beta)$	verossimilhança: produto das probabilidades corretas
$\sigma'(z)$	derivada: $\sigma(z)(1 - \sigma(z))$
∇	nabla, operador gradiente
α	taxa de aprendizado (passo)
$\Leftrightarrow$	“equivale a”
∞	infinito
$z = 0$	fronteira de decisão (p = 0,5)

Leitura: documentação oficial

NumPy

numpy.org/doc/stable

numpy.exp

numpy.clip

numpy.matmul (operador @)

Matplotlib

matplotlib.org/stable

pyplot.scatter

pyplot.plot

Scikit-learn

LogisticRegression

Só para comparação. Por padrão aplica regularização L2 (C = 1,0); os coeficientes não coincidem com os nossos.